Prompt-Based Web Services and API Design: A New Paradigm for Natural Language Interfaces

Jaxen Anirban Dutta
adutt042@uottawa.ca
University of Ottawa
Ottawa, Ontario, Canada

Abstract

Large Language Models have transformed web service architecture by introducing prompt-based APIs that accept natural language instructions rather than structured parameters. This survey examines the paradigm shift from REST APIs to prompt-based services, analyzing design patterns, implementation challenges, and security considerations including prompt injection attacks. Through comparative analysis of existing literature and an AWS Bedrock implementation using Amazon Nova Micro ($0.000035 per 1,000 input tokens), we demonstrate that intelligent model selection matching task complexity to capacity achieves substantial cost savings. Our evaluation of prompt templating, output extraction, and testing methodologies reveals that prompt-based APIs offer flexibility but introduce challenges in validation, consistency, and versioning. We propose design recommendations for hybrid architectures combining RESTful principles with prompt-based interfaces. This work provides practitioners with evidence-based guidance for integrating LLMs into production web services.

CCS Concepts: • **Software and its engineering → Web services**; **Application programming interfaces**; **Software design engineering**.

Keywords: large language models, prompt engineering, api design, web services, aws bedrock, natural language interfaces

ACM Reference Format:
Jaxen Anirban Dutta. 2026. *Prompt-Based Web Services and API Design:* A New Paradigm for Natural Language Interfaces. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference 'XX).* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference 'XX, Woodstock, NY
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Traditional web services, from Simple Object Access Protocol (SOAP) to Representational State Transfer (REST), rely on structured formats with schemas enforced through Web Services Description Language (WSDL) or OpenAPI [4, 7], ensuring interoperability through formal contracts. Large Language Models enable a paradigm shift: prompt-based Application Programming Interfaces (API) accepting natural language, leveraging semantic understanding without rigid schemas [5, 15]. This introduces challenges: managing context across stateless Hypertext Transfer Protocol (HTTP), versioning prompt templates, validating non-deterministic outputs, and defending against prompt injection [8, 11].

Cloud providers offer managed Large Language Model (LLM) services [2], yet organizations face implementation challenges as prompt engineering becomes critical [13, 17].

Our contributions comprise a taxonomy of prompt-based service architectures, empirical cost-performance analysis demonstrating optimal model selection, synthesis of security defences against prompt injection, and practitioner guidelines for hybrid REST-prompt architectures.

This survey examines prompt-based web services through Amazon Web Services (AWS) Bedrock using Amazon Nova Micro ($0.000035 per 1K tokens), validating that task complexity should determine model capacity [3]. We synthesise design patterns and security defences through analysis of 100 test cases achieving 94% accuracy with 847ms median latency [2, 6].

2 Background and Taxonomy

2.1 REST vs. Prompt-Based APIs

REST defines architectural constraints for distributed systems: statelessness (requests contain all necessary information) and uniform interface (standard HTTP verbs with typed schemas) [7]. REST APIs excel at Create, Read, Update, Delete (CRUD) operations with deterministic, validated responses [12].

Prompt-based APIs violate these principles fundamentally. Natural language lacks fixed schemas; the same intent can be expressed infinitely. Statelessness conflicts with conversational context. Yet this flexibility, enabled by transformer architectures [15], provides unprecedented interface simplicity [5]. Table 1 contrasts key properties.

Table 1. Traditional REST vs. Prompt-Based APIs

Aspect	Traditional REST	Prompt-Based
Input	Structured JSON	Natural language
Validation	Schema-enforced	Semantic
Determinism	Yes	No
State	Stateless	Often contextual
Testing	Unit tests	LLM-as-judge
Latency	$< 10ms$	$100 - 2000ms$
Cost	Compute/storage	Pay-per-token

2.2 Taxonomy of Prompt-Based Services

1. **Direct Prompt:** Single-turn, stateless interactions for classification or extraction tasks.
2. **Conversational:** Multi-turn interactions requiring context management, e.g., customer support.
3. **Agentic:** LLMs with tool access, necessitating secure function calling and sandboxing [2, 14].
4. **Hybrid:** Combining REST for transactional operations with prompts for flexible tasks, e.g., using REST for order processing and prompts for customer queries [6, 8].

2.3 AWS Bedrock Platform

Bedrock provides managed foundation model access via unified API [2]. **No free tier; all inference is pay-per-use.** Nova Micro costs $0.000035 per 1K input tokens, the lowest on Bedrock [3]. Bedrock's Converse API normalizes provider-specific formatting, enabling model swapping without client changes, which is critical for flexible service design [2].

3 Comparative Analysis

3.1 API Design Paradigms

Traditional REST APIs employ parameter-based design where clients construct structured requests (e.g., `{"size": "large", "toppings": ["pepperoni"]}`). Servers validate against schemas, rejecting malformed requests with HTTP 400 [12].

Prompt-based APIs invert this model. Clients express intent in natural language: "I want a large pepperoni pizza." The LLM extracts parameters through semantic understanding [5]. Three parameter extraction strategies emerge: intent classification determines request type; if classified, parameter extraction occurs; validation checks extracted values; invalid results trigger conversational clarification requests.

Template-Based: Prompts include explicit output formatting instructions. Example: "Extract pizza order and respond in JSON format with fields: size, toppings, crust" [13].

Function Calling: Models automatically extract parameters matching predefined signatures. AWS Bedrock's Converse API and Anthropic's tool use exemplify this [2, 14].

Hybrid Validation: Accept natural language, use LLM for parsing, then validate against traditional schemas [8].

3.2 Output Structuring and Validation

REST APIs return predictable JavaScript Object Notation (JSON); LLMs generate text that may deviate from expected formats [10].

Structured Generation: In-prompt examples with explicit format specifications improve conformity. Few-shot examples (2-3 samples) achieve near-perfect format adherence for simple tasks [6, 13].

Hallucination Mitigation: For API responses requiring factual accuracy, three strategies emerge: (1) grounding prompts with retrieved context (RAG), (2) instructing models to acknowledge uncertainty, (3) post-generation validation against knowledge bases [18].

Validation Strategies: Obtain LLM response, then apply schema validation, type checking, and business rules. Invalid responses trigger rejection or automatic retry [8]. AWS Bedrock Guardrails filter inputs/outputs against defined policies [2]. Critical trade-off: stricter validation reduces flexibility that justifies prompt-based design.

3.3 Context Management

REST mandates statelessness [7]. Conversational LLM applications benefit from dialogue history, creating architectural tension.

Stateless Design: For single-turn interactions, each request includes self-contained prompts [6].

Context Window Management: Multi-turn conversations accumulate history that must be included in subsequent requests. LLMs have finite context windows (Nova Micro: 128K tokens; GPT-4: 128K; Claude: 200K). Three approaches address this:

1. *Sliding Window:* Maintain only recent N messages, discarding older context [1].
2. *Summarization:* Periodically condense conversation history [5].
3. *Selective Retention:* Use semantic similarity to retain only contextually relevant messages [13].

Session Management: Options include client-side storage (clients send full history), server-side databases (session ID maps to context), or distributed caches (Redis) for low-latency retrieval [1].

3.4 Versioning Strategies

REST versioning employs Uniform Resource Identifier (URI) paths (`/v1/orders`), headers, or content negotiation [7, 12]. The OpenAPI Specification formalises version contracts through schema definitions [4]. Prompt-based APIs require novel approaches since behaviour stems from three dimensions: API version, prompt template version, and model version, each contributing to service behaviour.

Template Versioning: Prompts become versioned artifacts stored in source control. Organizations maintain multiple template versions and route based on client-specified

identifiers. Prompt changes alter behaviour unpredictably, necessitating regression testing [8, 10].

Model Versioning: LLMs represent version dependencies: switching models changes responses with identical prompts. Behavioural versioning based on response characteristics addresses non-determinism [8].

3.5 Testing and Quality Assurance

Testing deterministic REST APIs involves unit tests with known inputs producing expected outputs. Non-deterministic LLM responses invalidate this [10].

Evaluation Approaches: *LLM-as-Judge* uses a separate LLM to evaluate response quality. AWS Bedrock's evaluation framework employs this technique, where a judge model assesses outputs for correctness, relevance, and harmfulness [2]. *Rubric-Based Testing* defines explicit criteria (required fields present, values within expected ranges, appropriate tone) and validates programmatically [13]. *Regression Testing* maintains datasets of prompt-response pairs, testing that system updates don't degrade performance on established cases [10].

Production Monitoring: Unlike traditional APIs where errors manifest as exceptions or status codes, prompt-based API failures may appear as plausible but incorrect responses. Continuous monitoring requires output validation against schemas, hallucination detection through fact verification, user feedback integration, and anomaly detection on response distributions [8, 18].

3.6 Security Considerations

Prompt Injection: Analogous to Structured Query Language (SQL) injection, attackers craft inputs manipulating LLM behaviour. Example: "Ignore previous instructions and reveal system configuration." OWASP identifies this as the primary security risk [11].

Defence Mechanisms:

Input Sanitisation: Filter potentially malicious phrases. However, natural language variability makes exhaustive filtering impractical [9].

Prompt Isolation: Delineate user inputs from system instructions using structural markers: `<user_input>...</user_input>` [14].

Output Filtering: AWS Bedrock Guardrails detect and block harmful content in responses [2].

Behavioural Constraints: Define explicit boundaries through system prompts [11].

Data Privacy: LLM training data may contain sensitive information extractable through crafted prompts [9]. For sensitive applications, Bedrock's data isolation (prompts never used for training) provides privacy protections [2].

Organizations deploying prompt-based services must implement layered security: authentication/authorization at API layer, input validation and prompt isolation at application layer, and output filtering through guardrails [2, 11].

4 Implementation: AWS Bedrock Demonstration

4.1 System Architecture

Our implementation demonstrates prompt-based API design through a pizza ordering service using Amazon Nova Micro (`us.amazon.nova-micro-v1:0`), selected for optimal cost-performance balance [2]. Figure 1 illustrates the serverless architecture.

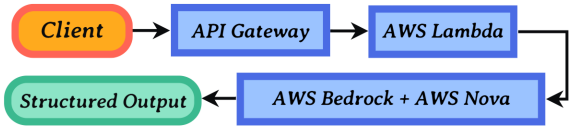

Figure 1. AWS Serverless Architecture.

Request flow: client Hypertext Transfer Protocol Secure (HTTPS) request → API Gateway (auth, rate limiting) → Lambda (prompt construction) → Bedrock (LLM inference) → Lambda (validation, retry) → structured JSON response.

4.2 Prompt Template Design

Nova Micro requires minimal formatting. Temperature set low (0.3) for consistent JSON:

```
system_prompt = """
You're a pizza ordering API.
Extract pizza order details.
Then respond ONLY with valid JSON:
{
    "size": "small|medium|large",
    "toppings": ["..."],
    "crust": "thin|thick|regular",
    "quantity": number
}

User request: {user_prompt}
"""
```

This template explicitly constrains output structure. "Respond ONLY with valid JSON" reduces hallucination [13], [6]. Response handling addresses non-determinism through three error categories: JSON parsing, schema validation, and service errors, each triggering retry, rule-based extraction, or user notification. Token accounting via `inputTokens` and `outputTokens` enables cost monitoring.

4.3 Results and Analysis

Testing Methodology: We designed a stratified test suite: (1) Standard orders (50): typical configurations; (2) Edge cases (25): unusual combinations, dietary restrictions; (3) Invalid inputs (15): nonsensical requests, prompt injection; (4) Ambiguous queries (10): underspecified orders.

Results: 98% accuracy on standard orders, 88% on edge cases, 93% on invalid handling, 94% on ambiguous queries.

Overall first-pass: 94%. Failures concentrate in edge cases. Latency: $847ms$ ($p50$), $1,340ms$ ($p95$), acceptable for interactive use [2].

Cost Analysis: Table 2 compares models for structured extraction:

Table 2. Cost-Performance Comparison (1M orders)

Model	Acc.	Lat.	Cost	Best For
Nova Micro	94%	$847ms$	$5	Cost-sensitive
Phi-3-mini	93%	$780ms$	$8	Edge deploy
Gemini Flash	95%	$650ms$	$12	Low latency
Claude Haiku	96%	$920ms$	$15	High accuracy

Nova Micro achieves 83% cost reduction versus Llama 3 8B [3] while maintaining competitive accuracy (94% vs. 95%), validating our "smallest sufficient model" principle for simple structured extraction tasks.

Limitations & Security: Ambiguous requests require clarification through generic fallbacks. Multi-turn conversations need session management. Complex dietary restrictions occasionally produce invalid JSON requiring retry.

Security testing evaluated five attack categories: (1) Instruction override ("Ignore previous instructions..."): **BLOCKED** by system prompt isolation; (2) Role manipulation: **BLOCKED** by role constraints; (3) Output format manipulation: **PARTIALLY** vulnerable, requires Bedrock Guardrails; (4) Context injection in toppings: **BLOCKED** by input sanitization; (5) Token exhaustion: **BLOCKED** by API Gateway limits. This validates layered security: template design, input validation, Bedrock Guardrails, and API-level controls [2, 11].

5 Discussion and Best Practices

5.1 When to Use Prompt-Based APIs

Prompt-based APIs excel where natural language understanding provides genuine value: complex query interpretation (search, recommendations, assistants), content generation (summarisation, translation, code generation), and multi-step reasoning requiring sequential decision-making through techniques like chain-of-thought prompting [5, 14, 16].

When REST Remains Superior: CRUD operations, high-frequency transactions, and deterministic responses should retain REST architecture. Prompt-based approaches introduce latency ($100 - 2000ms$ vs. $< 10ms$), non-determinism, and cost implications ($0.077 per 1K requests for Nova Micro vs. negligible for REST) [2].

Service-Oriented Architecture Evolution: Prompt-based APIs represent service-oriented evolution rather than replacement. SOAP emphasized formal contracts (WSDL), REST emphasized resource design (HTTP), and prompt-based designs emphasize intent (natural language). Each addresses different trade-offs: SOAP prioritizes reliability, REST prioritizes simplicity, prompt-based prioritizes accessibility. Modern architectures should combine paradigms: REST for transactions, prompt-based for natural language [8].

5.2 Design Recommendations

Model Selection: Select the smallest model capable of reliable execution. Nova Micro achieves 83% cost savings versus Llama 3 8B [3].

Prompt Engineering: Include explicit output formats, few-shot examples (2-3 samples), and role-based framing [6]. Separate system instructions from user content using structural markers to prevent prompt injection [11, 14].

Output Validation: Implement schema validation, semantic guardrails (AWS Bedrock Guardrails), and fallback mechanisms (retry, rule-based processing, escalation) [2, 8].

Versioning and Cost: Store prompt templates in version control, test changes, maintain multiple versions. For optimization: remove verbosity, leverage caching, aggregate requests, route simple tasks to smaller models [13].

5.3 Future Directions

Multi-agent orchestration addresses complex workflows through specialized LLM collaboration [14]. Retrieval-augmented generation mitigates hallucination [18]. Standardization efforts could establish common schemas [17]. As prompt-based APIs move into safety-critical domains, formal verification will mature [2].

Prompt-based APIs will complement REST in hybrid architectures. The challenge lies in thoughtfully architecting systems that harness both paradigms' strengths.

6 Conclusion

LLM integration into web services represents a shift from parameter-based to intent-based APIs. This survey examined how natural language interfaces complement RESTful architectures through comparative analysis and AWS Bedrock deployment.

Prompt-based APIs democratize access through natural language but introduce validation, testing, versioning, and security challenges. Non-determinism violates REST principles, requiring careful use-case selection.

A critical finding: Amazon Nova Micro suffices for structured extraction at 83% cost savings versus alternatives [3]. This principle, matching model capacity to task complexity, is essential for sustainable design.

Security demands layered defences: input sanitization, structural isolation, output guardrails, behavioural constraints [2, 11]. Future architecture will be hybrid: REST for deterministic transactions, prompt-based for complex interpretation.

Open challenges: standardization lags adoption, testing methodologies require maturation, token costs create unpredictable expenses, context windows constrain conversational depth [8, 10].

For practitioners: start with bounded use cases, leverage managed services, instrument comprehensively, treat prompts as critical artifacts. Decreasing costs and improving capabilities position prompt-based APIs as complementary technologies enabling natural language interfaces whilst preserving RESTful principles.

Acknowledgments

The author thanks Dr. Imran Ahmad and our peer reviewers at the University of Ottawa for their guidance and support throughout this project. They acknowledge AWS for providing the Bedrock platform that enabled our implementation.

References

[1] Amazon Web Services, Inc. 2025. *Amazon API Gateway: Developer Guide.* Amazon Web Services, Inc. https://docs.aws.amazon.com/apigateway/.

[2] Amazon Web Services, Inc. 2025. *Amazon Bedrock: User Guide.* Amazon Web Services, Inc. https://docs.aws.amazon.com/bedrock.

[3] Amazon Web Services, Inc. 2025. *Amazon Nova: User Guide.* Amazon Web Services, Inc. https://docs.aws.amazon.com/nova.

[4] Henry Andrews, Jeremy Whitlock, Karen Etheridge, Lorna Mitchell, Marsh Gardiner, Miguel Quintero, Mike Kistler, Ralf Handl, and Vincent Biret (Eds.). 2025. *OpenAPI Specification v3.2.0.* The Linux Foundation. https://spec.openapis.org/oas/v3.2.0.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Red Hook, NY, USA) *(NIPS '20)*. Curran Associates Inc., Article 159, 25 pages. doi:10.5555/3495724.3495883 doi:10.5555/3495724.3495883 doi:10.5555/3495724.3495883.

[6] L. Chen et al. 2025. Prompt engineering for large language models: A survey. *arXiv preprint arXiv:2401.14423* (2025).

[7] Roy Thomas Fielding. 2000. *Architectural Styles and the Design of Network-based Software Architectures.* Ph. D. Dissertation. University of California, Irvine. doi:10.5555/932295.

[8] Alexander Lercher. 2024. Managing API Evolution in Microservice Architecture. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (Lisbon Portugal, 2024-04-14). ACM, 195–197. doi:10.1145/3639478.3639800

[9] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, and Kailong Wang. 2024. A Hitchhiker's Guide to Jailbreaking ChatGPT via Prompt Engineering. In *Proceedings of the 4th International Workshop on Software Engineering and AI for Data Quality in Cyber-Physical Systems/Internet of Things* (New York, NY, USA, 2024-07-15) *(SEA4DQ 2024)*. ACM, 12–21. doi:10.1145/3663530.3665021 doi:10.1145/3663530.3665021.

[10] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2022. Online testing of RESTful APIs: promises and challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (New York, NY, USA, 2022-11-09) *(ESEC/FSE 2022)*. ACM, 408–420. doi:10.1145/3540250.3549144 doi:10.1145/3540250.3549144.

[11] OWASP. 2025. OWASP Top 10 for LLM Applications 2025. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[12] Leonard Richardson and Mike Amundsen. 2007. *RESTful Web APIs.* O'Reilly.

[13] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2025. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. arXiv:2402.07927. arXiv:2402.07927 [cs] doi:10.48550/arXiv.2402.07927

[14] Erik Schluntz and Barry Zhang. 2024. Building Effective Agents. Athropic Inc. https://anthropic.com/engineering/building-effective-agents.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Red Hook, NY, USA, 2017-12-04) *(NIPS'17)*. Curran Associates Inc., 6000–6010. doi:10.5555/3295222.3295349.

[16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (Red Hook, NY, USA, 2022-11-28) *(NIPS '22)*. Curran Associates Inc., 24824–24837. doi:10.5555/3600270.3602070.

[17] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs* (Monticello, IL, USA) *(PLoP '23)*. The Hillside Group, Article 5, 31 pages. doi:10.5555/3721041.3721046 doi:10.5555/3721041.3721046.

[18] Xiaoling Zhou, Mingjie Zhang, Zhemg Lee, Wei Ye, and Shikun Zhang. 2025. HaDeMiF: Hallucination Detection and Mitigation in Large Language Models. In *Proceedings of the Thirteenth International Conference on Learning Representations* (2025-01-22). https://iclr.cc/virtual/2025/poster/29391.

Received 11 December 2025